

Adaptive Reasoning Budgeting for LLMs

Atharva Chaudhari (ac151)

Abstract

Large language models often use the same amount of reasoning for every question, even though not all questions need the same amount of thinking. Easy questions may be solved with a short reasoning process, while harder questions may need more thinking steps. Using a fixed reasoning budget can therefore waste computation on simple questions and still be insufficient for difficult ones.

This work studies dynamic reasoning-budget allocation to solve problems. The goal is to train a controller that decides how many reasoning tokens a language model should use for each question. A two-stage setup is implemented. First, the model generates reasoning under a selected token budget. Then it produces the final answer from that reasoning. To learn budget selection, this work combines three ideas, which are oracle budget search, supervised warm-start, and reinforcement learning. Oracle search finds a good budget for each training question by testing several candidate budgets and choosing the one with the best reward. Supervised warm start teaches the controller to imitate these oracle budgets. Reinforcement learning then refines the controller to jointly optimize answer correctness and token efficiency. The best model achieves 80% accuracy on the GSM8K test set while using only around 185 tokens on average, using 39 fewer tokens per question, compared to the fixed 224-token baseline, which scored 79%, demonstrating that a policy network trained with PPO can learn to allocate reasoning budgets adaptively rather than applying a same-size token policy to every question.

1 Introduction

Large language models have shown strong performance on reasoning tasks such as math word problems, commonsense questions, and

step-by-step decision making. A common way to improve performance is to let the model generate intermediate reasoning before giving the final answer. This is called chain-of-thought reasoning Wei et al. (2022). Although, this helps to increase accuracy, it also increases computation because the model may generate many reasoning tokens before producing the answer.

A simple approach is to give every question the same reasoning budget. For example, the model may always be allowed to think for 160 or 220 tokens. This approach is easy to implement, but it has an important weakness, which is that different questions have different difficulty levels. A short arithmetic question may not need long reasoning, while a multi-step word problem may require much more computation. Because of this, a fixed budget can be inefficient. If the budget is too large, the model wastes tokens on easy questions. If the budget is too small, the model may fail on hard questions.

Instead of forcing the same number of reasoning tokens for all inputs, a policy network can predict an appropriate budget from the question itself. The larger language model remains the solver, while the policy network acts as a controller that decides how many thinking tokens the solver should use.

The approach is evaluated on GSM8K (Cobbe et al., 2021) and compared with fixed-budget baselines. Results show that the controller can learn non-constant, question-dependent reasoning budgets instead of collapsing to one fixed value. The token count in this work refers to subword tokens produced by the model tokenizer. Subword tokenization is chosen

because character-level tokens result in very long sequences and higher computational cost, while word-level tokens suffer from a large vocabulary size and poor handling of unseen words. Subword tokens provide a balanced approach by efficiently representing text while maintaining good generalization.

2 Related Work

The idea of using reinforcement learning to control how much computation a model uses started with work on adaptive computation. Graves (2016) introduced Adaptive Computation Time, where a neural network learns how many computational steps to use for each input. This work showed that not all inputs require the same amount of processing, and models can benefit from dynamically adjusting computation. Later, Wang et al. (2018) extended this idea using reinforcement learning in SkipNet, where the model learns to skip unnecessary layers to reduce computation. Similarly, Banino et al. (2021) proposed PonderNet, which allows a model to decide when to stop computing using a learned halting probability. These works are important because they establish that computation can be treated as a decision-making problem, but they operate at the level of network layers or hidden states and do not address reasoning tokens in language models.

With the rise of large language models, reinforcement learning became widely used for improving output quality and alignment. Ouyang et al. (2022) introduced Reinforcement Learning from Human Feedback, where a reward model trained from human preferences is used to optimize the language model. Bai et al. (2022) further extended this approach by using AI-generated feedback instead of human annotations, making the process more scalable. These methods demonstrate that reinforcement learning is effective for controlling model behavior. However, they focus mainly on improving helpfulness, safety, and correctness of responses, and they do not explicitly consider how much reasoning or

how many tokens should be used during inference.

More recent work has applied reinforcement learning directly to reasoning tasks. Zelikman et al. (2022) proposed STaR, where models improve reasoning by learning from their own generated rationales. Lightman et al. (2023) showed that supervising intermediate reasoning steps can further improve performance, highlighting the importance of reasoning processes rather than just final answers. In addition, Shao et al. (2024) introduced DeepSeekMath, which uses reinforcement learning to improve mathematical reasoning performance, and DeepSeek-AI (2025) demonstrated that reinforcement learning can lead to strong reasoning capabilities in large models. However, they typically assume a fixed reasoning length and do not address the efficiency problem of how many reasoning tokens should be generated.

Yang et al. (2025) introduced a dynamic early-exit strategy that allows models to stop reasoning when sufficient confidence is reached, reducing unnecessary computation.

Recent research has increasingly focused on controlling the reasoning length of large language models to improve efficiency while maintaining performance. Important work by Aggarwal and Welleck. (2025) explores how reinforcement learning can be used to directly control how long a model spends on reasoning. Instead of allowing the model to generate an arbitrary number of reasoning tokens, their approach introduces a learning signal that balances correctness with computational cost. The model is rewarded not only for producing accurate answers but also for using fewer reasoning steps when possible. This allows the system to learn an optimal trade-off between accuracy and efficiency. But the problem with this model is that the same language model is responsible for both generating the reasoning steps and implicitly deciding how long to continue reasoning. The reinforcement learning

objective directly modifies the model’s internal behavior so that it learns to stop earlier or continue longer depending on the reward. In other words, the model is jointly learning *what to think* and *how long to think* within a single set of parameters. Because of this, the decision about reasoning length is not explicitly separated or modeled as an independent problem. There is no dedicated component that first analyzes the input and predicts a suitable reasoning budget before the reasoning begins.

In this work, three learning algorithms, namely policy gradient, Proximal Policy Optimization (PPO), and Direct Preference Optimization (DPO), are explored to train a policy network that determines the number of reasoning tokens required for each input question.

3. Methodology and Results

3.1 System Overview

This work proposes a difficulty-aware adaptive reasoning budget allocation framework evaluated across 10 model configurations spanning two base language models and three types of training algorithms. All configurations share the same two-stage inference pipeline.

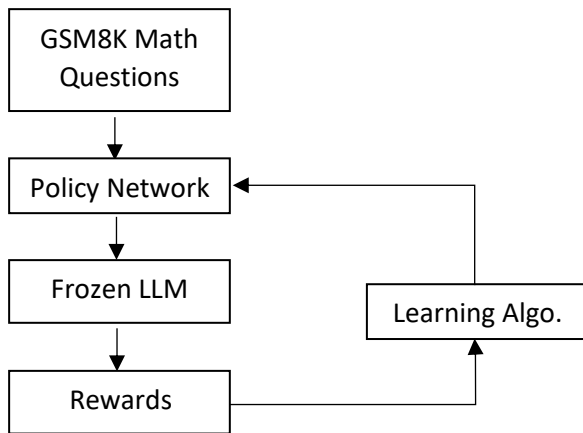


Fig 1. Methodology

In the first stage, the frozen language model is given a question and a token budget B and generates a step-by-step reasoning chain

within that token limit. In the second stage, the same model reads the generated reasoning and produces a final answer. A small trainable network called the budget policy network determines the value of B for each question before reasoning begins.

The budget range is determined empirically based on observed LLM performance across configurations. The lower bound is set to ensure sufficient reasoning capacity for the base model, while the upper bound is chosen based on the typical length of complete reasoning chains for GSM8K problems. The exact range varies across configurations.

The 10 configurations are organized into three groups. Models 1 through 5 use policy gradient methods, specifically REINFORCE and Actor-Critic (Konda and Tsitsiklis, 2000), on the Qwen2.5-1.5B-Instruct model. Models 6 through 9 upgrade the training algorithm to Proximal Policy Optimization (PPO), with Models 6 through 8 continuing on the 1.5B model and Model 9 scaling to Qwen2.5-7B-Instruct. The final configuration, Model 10, replaces PPO entirely with Direct Preference Optimization (DPO) on the 7B model, exploring a contrastive preference-based training approach as an alternative to reward-signal-driven reinforcement learning.

The base language model remains completely frozen throughout all configurations. Only the budget policy network is trained. All experiments are conducted on the GSM8K benchmark using questions drawn from the canonical train and test splits in order. Model 1 uses 80 training and 20 test questions. Model 2 expands this to 200 training and 50 test questions. From Model 3 onwards, all configurations use 300 training and 100 test questions throughout.

3.2 Question Representation

Each question is encoded as a fixed-dimensional feature vector combining a pre-

trained sentence embedding with handcrafted difficulty features. The sentence embedding is produced by the frozen model all-MiniLM-L6-v2, which maps each question to a 384-dimensional semantic vector capturing meaning and phrasing. The handcrafted features are appended directly to this embedding and cover surface-level difficulty signals such as token count, digit count, character length, sentence count, and binary keyword flags indicating arithmetic operation cues such as addition, subtraction, multiplication, and division, as well as domain-specific terms such as "total", "remaining", and "average". This feature extractor is frozen and not updated during training across all configurations.

The feature vector dimensionality evolves across model configurations as the handcrafted component is expanded. Models 1 through 4 use 29 handcrafted features for a total input dimension of 413. Models 5 through 7 expand this to 50 handcrafted features, giving a 434-dimensional vector. Models 8 through 10 add 12 further difficulty-aware features covering operation count, large number detection, multi-step sequence indicators, conditional logic, unit conversion, percentage and fraction complexity, entity count, and a composite difficulty score, bringing the total to 62 handcrafted features and a final input dimension of 446. The policy network input dimension is updated accordingly across configurations while all other components remain identical.

3.3 Budget Policy Network

The budget policy network is a small two-layer neural network with 256 neurons per layer and approximately 170,000 trainable parameters. It takes the question feature vector as input and outputs a single number representing the thinking token budget B for that question.

During training, the network samples budgets randomly around its predicted value to explore

different options and learn from the outcomes. During evaluation, it uses its best predicted value directly without any randomness.

Since every question has a different feature vector, the network produces a different budget for every question. This is what makes the system adaptive. Questions with more complex patterns in their features tend to receive larger budgets, while simpler questions receive smaller ones, as a direct result of what the network learned during training.

3.4 Language Model

Two base language models are used across the 10 models. Qwen2.5-1.5B-Instruct is used for Models 1 through 8 in the 1.5B experiments, while Qwen2.5-7B-Instruct is introduced in Model 9 and Model 10 to evaluate whether the adaptive budget approach scales to a larger and more capable base model.

Fixed budget baselines are evaluated on the same test set as the policy within each model, ensuring fair comparison; baseline numbers differ across models due to changes in test set size.

3.5 Model Configurations

Model 1

The first model trained the budget policy on 80 math questions for 2 rounds of training. The reward signal was straightforward; the policy earned 1 point for a correct answer and lost a small amount for every token it used

$$r = \text{correct} - 0.0005 \times \text{thinking_tokens_used}$$

Before training begins, four fixed budgets are tested on the held-out test set of 20 questions to establish reference points. A budget of 96 tokens achieves 10%, 128 tokens achieve 5%, and both 160 and 220 tokens achieve 20%, showing that more thinking tokens generally help up to a point, but the gains are inconsistent on this scale.

After training, the policy reaches 22% to 28% accuracy on training questions across two epochs. On the held-out test set, however, accuracy stays stuck at 15% throughout. The policy settles on allocating almost exactly 142 tokens to every single question, regardless of whether the question is simple or complex. The average budget on the test set is 142.05 tokens with zero spread across questions.

Two problems are identified with this outcome. First, 80 training examples are simply not enough for the policy to pick up on meaningful patterns that distinguish easy questions from hard ones. Second, because most sampled budgets lead to wrong answers, the reward signal is too noisy and sparse to give the policy anything useful to learn from.

Model 2

In this model, the first change adds a value network trained alongside the policy. Rather than updating the policy directly on the reward signal, the policy is updated on the advantage, which is the difference between the actual reward received and what the value network predicted the reward would be. This advantage-based update produces a cleaner and more stable gradient signal than raw reward alone. This approach is known as Actor-Critic (Konda and Tsitsiklis, 2000). The second change expands the training set from 80 to 200 questions and extends training from 2 to 5 epochs. The third change replaces the binary correct-or-wrong reward with a softer version that awards partial credit when the predicted answer is numerically close to the correct one.

$$answer_score = \max(exact_match, 1 - |pred - gold| / \max(|gold|, 1.0))$$

$$r = answer_score - 0.0002 \times thinking_tokens_used$$

Method	Avg Tokens	Test Accuracy
Fixed 96 Tok	96	24%
Fixed 128 Tok	128	24%
Fixed 160 Tok	160	30%

Fixed 220 Tok	220	36%
Model 2	142	30%

Table 1

After training, test accuracy reaches 30%. However, the budget distribution on the test set collapses completely, with an average of 141.88 tokens, a standard deviation of only 0.71, and a range of just 140 to 143 tokens across all test questions. The policy had learned to ignore the question features and predict the same safe budget every time. The soft reward is identified as the primary cause. Because partial credit is available for near-correct answers at any budget level, a fixed mid-range budget consistently collects enough reward to appear locally optimal.

Model 3:

The third configuration introduces a two-phase training to address the budget collapse problem observed in Models 1 and 2. Rather than starting the policy from random weights and relying entirely on reinforcement learning to discover good budgets from scratch, a supervised warm-start phase is introduced to give the policy a meaningful starting point before any RL training begins.

In the warm-start phase, the frozen LLM is run at 11 candidate budgets ranging from 64 to 220 tokens on every training question. For each question, the budget that produces the highest reward is recorded as the oracle label. The policy is then trained for 15 epochs via supervised regression to predict these oracle labels, using mean squared error loss. This gives the policy a strong and informed starting point that reflects actual LLM performance across the budget range before any exploration begins.

$$answer_score = \max(exact_match, 1 - |pred - gold| / \max(|gold|, 1.0))$$

$$r = answer_score - 0.0001 \times thinking_tokens_used$$

Following warm-start, the policy is fine-tuned for 5 epochs using the same Actor-Critic

approach as Model 2, with 25% random budget exploration mixed in during the first 2 epochs to prevent the policy from collapsing back to a fixed strategy.

Method	Avg Tokens	Test Accuracy
Fixed 96 Tok	96	27%
Fixed 128 Tok	128	33%
Fixed 160 Tok	160	49%
Fixed 220 Tok	220	53%
Model 3	103	30%

Table 2

The warm-started policy achieves 30% accuracy while using only 102.98 tokens on average, with budgets distributed across a range of 85 to 139 tokens and a standard deviation of 11.18. The policy reaches comparable accuracy to the fixed 128-token baseline while using 25 fewer tokens on average, suggesting the policy has learned to allocate more tokens to harder questions and fewer to easier ones.

However, the 5 epochs of RL fine-tuning that follow do not improve test accuracy beyond 30%, which remains stuck across all 5 epochs. The soft reward continues to create a ceiling where a consistent mid-range strategy collects enough partial credit to appear locally optimal.

Model 4

Model 4 keeps the same two-phase structure as Model 3 but makes three changes

The first change is the reward function. Soft partial credit is removed and replaced with a hard binary signal plus an explicit bonus for correct answers,

$$r = \text{correct} + 0.25 \times \text{correct} - 0.00005 \times \text{thinking_tokens_used}$$

A correct answer now earns roughly 1.25 minus a small token penalty. A wrong answer earns only the negative token penalty. This larger gap between right and wrong gives the policy a much cleaner training signal.

The second change is the exploration floor. In Model 3, random exploration during RL warmup could inject budgets as low as 64 tokens, where the LLM almost always fails. In Model 4, the minimum random exploration budget is raised to 128 tokens, keeping all exploratory rollouts in a range where correct answers are at least plausible.

The third change is checkpoint selection. Both a best-by-reward and a best-by-accuracy warm-start checkpoint are saved separately, and the best-by-accuracy version is loaded before RL begins.

Method	Avg Tokens	Test Accuracy
Fixed 96 Tok.	96	27%
Fixed 128 Tok.	128	33%
Fixed 144 Tok	144	38%
Fixed 160 Tok	160	49%
Fixed 176 Tok	176	54%
Fixed 192 Tok	192	52%
Fixed 220 Tok	220	53%
Model 4	116	33%

Table 3

But the 5 RL epochs again produce no improvement. Test accuracy stays at exactly 33% across all 5 epochs with an unchanged average budget of 116 tokens.

Model 5

Model 5 keeps the same Actor-Critic update rule but makes four changes. The oracle label is changed to the minimum correct budget per question instead of the highest reward, directly teaching the policy to be efficient. The entropy coefficient is raised from 0.002 to 0.02 to prevent budget collapse. A supervised oracle blend loss is added to every RL step to keep the deterministic head learning. The token penalty is raised from 0.00005 to 0.003 to give the policy a real incentive to use fewer tokens on easy questions.

Despite these changes, the final policy achieves only 32% accuracy at 130 tokens, still not beating the fixed 128-token baseline

of 33%. The Actor-Critic update is still too noisy without any clipping, causing the policy to oscillate rather than converge.

The problem with Reinforce and Actor-Critic was that single-sample gradient updates are too noisy. One bad LLM rollout could push the policy in the wrong direction.

Model 6 and Model 7

Models 6 and 7 replace the Actor-Critic update with Proximal Policy Optimization (PPO).

The PPO clip constrains how much the policy can change per update by computing the ratio between new and old log probabilities and clamping it to the range 0.8 to 1.2 (Schulman et al., 2017). This prevents any single rollout from pushing the policy too far in the wrong direction, which was the core failure mode of all previous Actor-Critic models.

Models 6 and 7 share the same architecture: 15 warm-start epochs, value network pre-training on oracle rewards before PPO begins, oracle blend loss on the deterministic head during every PPO update, and 2 PPO update epochs per rollout. The only difference between them is the Oracle label strategy. Model 6 uses the highest reward budget as the oracle label, while Model 7 switches to the minimum correct budget to encourage efficiency.

Model 6 reaches 39% accuracy at 182 tokens after 10 PPO epochs, beating every fixed budget up to 176 tokens. Model 7 reaches 34% at 167 tokens. The minimum correct oracle in Model 7 does not help because 67% of training questions get labeled as 128 tokens, which skews the warm start too heavily toward short budgets and gives the policy a poor starting point.

Method	Avg Tokens	Test Acc.
Fixed 128 Tok	128	25%
Fixed 160 Tok	160	25%
Fixed 176 Tok	176	35%

Fixed 184 Tok	184	39%
Fixed 192 Tok	192	41%
Fixed 210 Tok	210	45%
Fixed 220 Tok	220	42%
Model 6	182	39%
Model 7	167	34%

Table 4

Model 8

In this model, the feature vector is expanded from 434 to 446 dimensions by adding 12 new difficulty-aware handcrafted features covering operation count, large number detection, multi-step cues, conditional logic, unit conversion, and a composite difficulty score. The budget range is also expanded from 128-220 to 96-256 tokens to give the policy more room to explore.

After 10 PPO epochs, the policy achieves 49% accuracy at 223 Avg. tokens, which range from 213 to 241, matching the fixed 256-token baseline of 50% while using fewer tokens on average.

Method	Avg Tokens	Accuracy
Fixed 96 Tok	96	22%
Fixed 128 Tok	128	25%
Fixed 160 Tok	160	24%
Fixed 192 Tok	192	39%
Fixed 256 Tok	256	50%
Model 8	223	49%

Table 5

After eight configurations with the 1.5B model, accuracy plateaued around 49% regardless of how the reward, oracle, or features were tuned. This raised a natural question: Can a stronger LLM break through this ceiling?

Model 9

Model 9 is upgraded from Qwen 2.5-1.5B to Qwen 2.5-7B. The impact of the larger model is immediately visible in the fixed budget baselines. The 7B model achieves 63% at 128 tokens and 74% at 160 tokens, compared to 25% and 24% for the 1.5B.

After warm-start epochs, the policy achieves 69% accuracy at 146 tokens, higher than any 1.5B configuration ever reached. PPO training then improves this further, with 80% accuracy at 185 tokens.

Method	Avg Tokens	Acc
Fixed 96 Tok	96	47%
Fixed 128 Tok	128	63%
Fixed 160 Tok	160	74%
Fixed 192 Tok	192	76%
Fixed 224 Tok	224	79%
Model 9	185	80%

Table 6

Model 9 is not just accurate, it is efficient. It allocates adaptively. Fixed budgets either overspend tokens on easy questions or underspend on hard ones. The PPO policy fixes both failure modes simultaneously.

Model 10

Model 10 explores whether Direct Preference Optimization (DPO) can achieve similar results with a much simpler training loop, no value network, no rollouts, and no advantage estimation.

DPO learns from preference pairs instead of rewards. For each training question, a preferred budget is defined as the minimum budget at which the 7B model answers correctly, and a rejected budget is the maximum budget at which it still answers incorrectly.

DPO fails for two reasons. First, only 74 of 300 training questions produce valid preference pairs, meaning 75% of the training data contributes no gradient signal. Second, without a reward signal to penalize wrong answers, there is no lower bound on budget reduction, causing the policy to collapse steadily toward shorter and shorter budgets until the LLM can no longer reason correctly. PPO avoids both problems by receiving a reward signal on every question every rollout, naturally preventing collapse.

4. Discussion

This work investigates whether a policy network can learn to allocate thinking token budgets per question rather than applying a uniform fixed budget, and whether doing so improves the accuracy-versus-tokens tradeoff on GSM8K.

The ten-model progression reveals that adaptive budget control is an engineering problem as much as an algorithmic one. Budget collapse, noisy advantage estimates, poor oracle label design, and unstable policy gradients each independently prevented convergence in earlier configurations. PPO with clipped updates, value network pre-training, oracle warm-start, and a difficulty-aware 446-dimension feature vector had to be working simultaneously before the system converged reliably. No single fix was sufficient alone.

DPO is appealing for its simplicity, but fails here for structural reasons; preference pairs can only be constructed for questions with both a correct and an incorrect oracle budget.

5. Limitations

The evaluation is restricted to 100 test questions from GSM8K, a grade-school arithmetic dataset with a narrow and homogeneous difficulty distribution that may not reflect the complexity of real-world reasoning tasks. The budget range of 96 to 256 tokens is short by the standards of modern chain-of-thought reasoning, where effective reasoning traces can span thousands of tokens. Finally, all results are reported on a single dataset and a single type of model family, and it remains unclear how well the findings generalize to other benchmarks, model architectures, or reasoning domains.

6. References

- Wei, Jason, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.
- Cobbe, Karl, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Graves, Alex. 2016. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*.
- Wang, Xin, Fisher Yu, Zi-Yi Dou, and Trevor Darrell. 2018. SkipNet: Learning dynamic routing in convolutional networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 409–424.
- Banino, Andrea, et al. 2021. PonderNet: Learning to ponder. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 646–655.
- Ouyang, Long, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 27730–27744.
- Bai, Yuntao, et al. 2022. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.
- Zelikman, Eric, et al. 2022. STaR: Self-taught reasoner for language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Lightman, Hunter, et al. 2023. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*.
- Shao, Zhihong, et al. 2024. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Yang, Chenxu, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. 2025. Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*.
- Aggarwal, Pranjal, and Sean Welleck. 2025. L1: Controlling how long a reasoning model thinks with reinforcement learning. *arXiv preprint arXiv:2503.04697*.
- Konda, Vijay R., and John N. Tsitsiklis. 2000. Actor-critic algorithms. In *Advances in Neural Information Processing Systems*, volume 13, pages 1008--1014. MIT Press.
- Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

AI Disclosure

In preparing this report, I used Claude (Anthropic, claude.ai) to help refine the Reference Section in ACL format. All experimental design, model implementation, training, results, and analysis are entirely my own original work. I take full responsibility for all content presented in this report.

Appendix A: Hyperparameter Summary

Hyperparameter	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
LLM	1.5B	1.5B	1.5B	1.5B	1.5B	1.5B	1.5B	1.5B	7B	7B
Algorithm	REINFORCE	Actor-Critic	Actor-Critic	Actor-Critic	Actor-Critic	PPO	PPO	PPO	PPO	DPO
Train Questions	80	200	300	300	300	300	300	300	300	300
Test Questions	20	50	100	100	100	100	100	100	100	100
Warm-start Epochs	0	0	15	15	10	15	15	10	10	10
RL/DPO Epochs	2	5	5	5	5	10	10	10	10	20
PPO Clip (ϵ)	—	—	—	—	—	0.2	0.2	0.2	0.2	—
PPO Update Epochs	—	—	—	—	—	2	2	2	2	—
DPO Beta	—	—	—	—	—	—	—	—	—	0.1
Policy LR	1e-3	1e-3	5e-4	5e-4	5e-4	3e-4	3e-4	3e-4	3e-4	5e-5
Value LR	—	1e-3	8e-4	8e-4	8e-4	8e-4	8e-4	8e-4	8e-4	—
Entropy Coef	0.01	0.01	0.002	0.002	0.02	0.02	0.02	0.02	0.02	—
Token Penalty	0.0005	0.0002	0.0001	0.00005	0.003	0.0005	0.0005	0.0005	0.0005	0.0005
Oracle Label	—	—	Highest reward	Highest reward	Min correct	Highest reward	Min correct	Min correct	Min correct	Min correct
Budget Range	96-256	96-256	64-220	64-220	96-256	128-220	128-220	96-256	96-256	96-256
Feature Dim	413	413	413	413	434	434	434	446	446	446